🎯 Objective

The goal of this room is to master **tcpdump**, a powerful command-line packet analyzer. I learned how to capture live network traffic, filter specific packets using expressions, and save captures into .pcap files for later analysis in tools like Wireshark.

🔍 Key Concepts Covered

1. *Why Tcpdump?*

Unlike Wireshark, tcpdump is lightweight and runs entirely in the CLI. This makes it the go-to tool for:

- Analyzing traffic on **headless servers** (no GUI).
- Performing quick "sanity checks" on network connectivity.
- Capturing data on low-resource IoT devices.

2. Essential Flags & Switches

I practiced using the core switches that control how tcpdump behaves:

- -i [interface]: Specifies which interface to listen on (e.g., eth0).
- -n: Disables name resolution (shows IP addresses instead of hostnames), making it much faster.
- -v, -vv, -vvv: Increases the level of **verbosity** (detail) shown for each packet.
- -c [number]: Captures only a specific number of packets and then exits.
- -X: Displays the packet content in both **Hex and ASCII**.

3. Reading and Writing PCAPs

I learned the workflow for professional traffic analysis:

- **Write to file:** tcpdump -i eth0 -w capture.pcap (Saves raw packets).

- **Read from file:** tcpdump -r capture.pcap (Processes a previously saved capture).

🛠️ Lab Walkthrough & Filtering Logic

Task: Using Packet Filters

The real power of tcpdump lies in its filtering syntax. During the lab, I used the following filters to isolate malicious or specific traffic:

1. **Filter by Host:** tcpdump host 10.10.1.5
2. **Filter by Port:** tcpdump port 80 (Isolating HTTP traffic).
3. **Complex Logic:** tcpdump src 10.10.1.5 and dst port 443
    1. *Analysis:* This captures only outgoing HTTPS traffic from a specific machine, which is useful for identifying "phone-home" behavior in malware.

Answer the questions below

What is the name of the library that is associated with tcpdump*?*

Answer: libpcap

Basic Packet Capture

Tcpdump is a tool for capturing network packets. Some essential commands for capturing packets:

- tcpdump -i INTERFACE: Captures packets on a specified network interface (e.g., -i eth0 for Ethernet or -i any for all interfaces).
- tcpdump -w FILE: Saves captured packets to a file, usually with a .pcap extension, for later analysis.
- tcpdump -r FILE: Reads and displays packets from a saved file.
- tcpdump -c COUNT: Limits capture to a specific number of packets.
- tcpdump -n / -nn: Avoids resolving IP addresses and ports to domain names and protocol names, making output quicker and easier to interpret.

Example:

```
user@TryHackMe$ sudo tcpdump -i ens5 -c 5 -n

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode

listening on ens5, link-type EN10MB (Ethernet), capture size 262144 bytes
```

08:55:18.989213 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags [P.], seq 2888580014:2888580210, ack 771262362, win 922, options [nop,nop,TS val 3216251159 ecr 33295823], length 196

08:55:18.989446 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags [P.], seq 196:424, ack 1, win 922, options [nop,nop,TS val 3216251159 ecr 33295823], length 228

08:55:18.989576 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags [P.], seq 424:620, ack 1, win 922, options [nop,nop,TS val 3216251159 ecr 33295823], length 196

08:55:18.989839 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags [P.], seq 620:816, ack 1, win 922, options [nop,nop,TS val 3216251159 ecr 33295823], length 196

08:55:18.989958 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags [P.], seq 816:1012, ack 1, win 922, options [nop,nop,TS val 3216251159 ecr 33295823], length 196

5 packets captured

6 packets received by filter

0 packets dropped by kernel

Consider the following examples:

- tcpdump -i eth0 -c 50 -v captures and displays 50 packets by listening on the eth0 interface, which is a wired Ethernet, and displays them verbosely.
- tcpdump -i wlo1 -w data.pcap captures packets by listening on the wlo1 interface (the WiFi interface) and writes the packets to data.pcap. It will continue till the user interrupts the capture by pressing CTRL-C.
- tcpdump -i any -nn captures packets on all interfaces and displays them on screen without domain name or protocol resolution.

Answer the questions below

What option can you add to your command to display addresses only in numeric format?

Answer: -n

Filtering Expressions

To filter packets based on criteria such as hosts, ports, or protocols:

- **Host Filtering**: tcpdump host HOSTNAME captures packets involving a specific host. Use src host HOST or dst host HOST to filter packets only from a source or to a destination.

```
user@TryHackMe$ sudo tcpdump host example.com -w http.pcap

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

16:49:02.482295 IP 192.168.139.132.49480 > 93.184.215.14.http: Flags [S], seq 3330895816, win 32120, options [mss 1460,sackOK,TS val 621343956 ecr 0,nop,wscale 7], length 0

16:49:02.635087 IP 93.184.215.14.http > 192.168.139.132.49480: Flags [S.], seq 2231582859, ack 3330895817, win 64240, options [mss 1460], length 0

16:49:02.635125 IP 192.168.139.132.49480 > 93.184.215.14.http: Flags [.], ack 1, win 32120, length 0

16:49:02.635491 IP 192.168.139.132.49480 > 93.184.215.14.http: Flags [P.], seq 1:131, ack 1, win 32120, length 130: HTTP: GET / HTTP/1.1

16:49:02.635580 IP 93.184.215.14.http > 192.168.139.132.49480: Flags [.], ack 131, win 64240, length 0
```

[...]

^C

13 packets captured

25 packets received by filter

0 packets dropped by kernel

- **Port Filtering**: tcpdump port PORT_NUMBER captures packets on a specific port. Use src port or dst port for finer filtering.

user@TryHackMe$ sudo tcpdump -i ens5 port 53 -n

[sudo] password for strategos:

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

17:26:33.591670 IP 192.168.139.132.47902 > 192.168.139.2.53: 47108+ A? example.org. (29)

```
17:26:33.591717 IP 192.168.139.132.47902 > 192.168.139.2.53: 5+ AAAA?
example.org. (29)

17:26:33.593324 IP 192.168.139.2.53 > 192.168.139.132.47902: 47108 1/0/0 A
93.184.215.14 (45)

17:26:33.593325 IP 192.168.139.2.53 > 192.168.139.132.47902: 5 1/0/0 AAAA
2606:2800:21f:cb07:6820:80da:af6b:8b2c (57)

[...]

^C

12 packets captured

12 packets received by filter

0 packets dropped by kernel
```

- **Protocol Filtering**: tcpdump PROTOCOL filters packets by protocol, like tcp, udp, icmp, etc.

```
user@TryHackMe$ sudo tcpdump -i ens5 icmp -n

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
```

```
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

18:11:00.624681 IP 192.168.139.132 > 93.184.215.14: ICMP echo request, id 47038, seq 1, length 64

18:11:00.781482 IP 93.184.215.14 > 192.168.139.132: ICMP echo reply, id 47038, seq 1, length 64

18:11:04.168792 IP 192.168.139.2 > 192.168.139.132: ICMP time exceeded in-transit, length 68

18:11:04.168815 IP 192.168.139.2 > 192.168.139.132: ICMP time exceeded in-transit, length 68

[...]

18:11:14.857188 IP 93.184.215.14 > 192.168.139.132: ICMP 93.184.215.14 udp port 33495 unreachable, length 68

^C

52 packets captured

52 packets received by filter
```

0 packets dropped by kernel

Logical Operators in Tcpdump:

- **and** (tcpdump host 1.1.1.1 and tcp): Captures packets matching both conditions.
- **or** (tcpdump udp or icmp): Captures packets meeting either condition.
- **not** (tcpdump not tcp): Captures all packets except those matching the condition.

Consider the following examples:

- tcpdump -i any tcp port 22 listens on all interfaces and captures tcp packets to or from port 22, i.e., SSH traffic.
- tcpdump -i wlo1 udp port 123 listens on the WiFi network card and filters udp traffic to port 123, the Network Time Protocol (NTP).
- tcpdump -i eth0 host example.com and tcp port 443 -w https.pcap will listen on eth0, the wired Ethernet interface and filter traffic exchanged with example.com that uses tcp and port 443. In other words, this command is filtering HTTPS traffic related to example.com.

For the questions from this task, we will read captured packets from the traffic.pcap file. As mentioned earlier, we use -r FILE to read from a packet capture file. To test

this, try tcpdump -r traffic.pcap -c 5 -n; it should display the first five packets in the file without looking up the IP addresses.

Remember that you can count the lines by piping the output via the wc command. In the terminal below, we can see that we have 910 packets with the source IP address set to 192.168.124.1. Please note that we add -n to avoid unnecessary delays in attempting to resolve IP addresses. In the example below, we didn't use sudo as reading from a packet capture file does not require root privileges.

```
user@TryHackMe$ tcpdump -r traffic.pcap src host 192.168.124.1 -n | wc

reading from file traffic.pcap, link-type EN10MB (Ethernet)

  910   17415  140616
```

Answer the questions below

How many packets in traffic.pcap *use the ICMP protocol?*

Answer: 26

```
user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap icmp -n | wc
```

reading from file traffic.pcap, link-type EN10MB (Ethernet)

26 358 2722

What is the IP address of the host that asked for the MAC address of 192.168.124.137?

Answer: 192.168.124.148

user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap arp and host 192.168.124.137

reading from file traffic.pcap, link-type EN10MB (Ethernet)

07:18:29.940761 ARP, Request who-has ip-192-168-124-137.eu-west-1.compute.internal tell ip-192-168-124-148.eu-west-1.compute.internal, length 28

07:18:29.940776 ARP, Reply ip-192-168-124-137.eu-west-1.compute.internal is-at 52:54:00:23:60:2b (oui Unknown), length 28

What hostname (subdomain) appears in the first DNS query?

Answer: mirrors.rockylinux.org

user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap port 53 -A

reading from file traffic.pcap, link-type EN10MB (Ethernet)

07:18:24.058626 IP ip-192-168-124-137.eu-west-1.compute.internal.33672 > ip-192-168-124-1.eu-west-1.compute.internal.domain: 39913+ A? mirrors.rockylinux.org. (40)

E..D..@.@.'...|...|....5.0z.....mirrors

rockylinux.org.....

Advanced Filtering

Advanced filtering uses conditions for more specific needs:

- **Packet Length**: Use greater LENGTH or less LENGTH to capture packets above or below a specified length.
- **Binary Operations**: Filters can be defined using binary operators on protocol headers. Example:
- tcp[tcpflags] == tcp-syn: Captures packets with only the SYN flag set.
- tcp[tcpflags] & tcp-ack != 0: Captures packets with at least the ACK flag set.

Before proceeding, it is worth visiting binary operations. A binary operation works on bits, i.e., zeroes and ones. An operation takes one or two bits and returns one bit.

Let’s explain in more depth and consider the following three binary operations: &, |, and !.

& (And) takes two bits and returns 0 unless both inputs are 1, as shown in the table below.

Input 1	Input 2	Input1 & Input 2
0	0	0
0	1	0
1	0	0
1	1	1

| (Or) takes two bits and returns 1 unless both inputs are 0. This is shown in the table below.

Input 1	Input 2	Input 1 | Input 2
0	0	0
0	1	1
1	0	1
1	1	1

! (Not) takes one bit and inverts it; an input of 1 gives 0, and an input of 0 gives 1, as shown in the table below.

Input 1	! Input 1
0	1
1	0

Header Bytes

The purpose of this section is to be able to filter packets based on the contents of a header byte. Consider the following protocols: ARP, Ethernet, ICMP, IP, TCP, and UDP. These are just a few networking protocols we have studied. How can we tell Tcpdump to filter packets based on the contents of protocol header bytes? (We will not go into details about the headers of each protocol as this is beyond the scope of this room; instead, we will focus on TCP flags.)

Using pcap-filter, Tcpdump allows you to refer to the contents of any byte in the header using the following syntax proto[expr:size], where:

- proto refers to the protocol. For example, arp, ether, icmp, ip, ip6, tcp, and udp refer to ARP, Ethernet, ICMP, IPv4, IPv6, TCP, and UDP respectively.
- expr indicates the byte offset, where 0 refers to the first byte.
- size indicates the number of bytes that interest us, which can be one, two, or four. It is optional and is one by default.

To better understand this, consider the following two examples from the pcap-filter manual page (and don't worry if you find them difficult):

- ether[0] & 1 != 0 takes the first byte in the Ethernet header and the decimal number 1 (i.e., 0000 0001 in binary) and applies the & (the And binary operation). It will return true if the result is not equal to the number

0 (i.e., 0000 0000). The purpose of this filter is to show packets sent to a multicast address. A multicast Ethernet address is a particular address that identifies a group of devices intended to receive the same data.

- ip[0] & 0xf != 5 takes the first byte in the IP header and compares it with the hexadecimal number F (i.e., 0000 1111 in binary). It will return true if the result is not equal to the (decimal) number 5 (i.e., 0000 0101 in binary). The purpose of this filter is to catch all IP packets with options.

Don’t worry if you find the above two examples complex. We included them so you know what you can achieve with this; however, fully understanding the above examples is not necessary to finish this task. Instead, we will focus on filtering TCP packets based on the set TCP flags.

You can use tcp[tcpflags] to refer to the TCP flags field. The following TCP flags are available to compare with:

- tcp-syn TCP SYN (Synchronize)
- tcp-ack TCP ACK (Acknowledge)
- tcp-fin TCP FIN (Finish)
- tcp-rst TCP RST (Reset)
- tcp-push TCP Push

Based on the above, we can write:

- tcpdump "tcp[tcpflags] == tcp-syn" to capture TCP packets with **only** the SYN (Synchronize) flag set, while all the other flags are unset.
- tcpdump "tcp[tcpflags] & tcp-syn != 0" to capture TCP packets with **at least** the SYN (Synchronize) flag set.
- tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0" to capture TCP packets with **at least** the SYN (Synchronize) **or** ACK (Acknowledge) flags set.

You can write your own filter depending on what you are looking for.

Answer the questions below

How many packets have only the TCP Reset (RST) flag set?

Answer: 57

```
user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap 'tcp[tcpflags] == tcp-rst' | wc -l

reading from file traffic.pcap, link-type EN10MB (Ethernet)

57
```

What is the IP address of the host that sent packets larger than 15000 bytes?

Answer: 185.117.80.53

```
user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap 'greater 15000' -n

reading from file traffic.pcap, link-type EN10MB (Ethernet)

07:18:24.967023 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq
2140876081:2140896901, ack 741991605, win 235, options [nop,nop,TS val
2226566282 ecr 3054280184], length 20820: H

TTP

07:18:25.778012 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq
1293616:1308884, ack 1, win 235, options [nop,nop,TS val 2226567095 ecr
3054280994], length 15268: HTTP

07:18:25.861724 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq
1378284:1397716, ack 1, win 235, options [nop,nop,TS val 2226567176 ecr
3054281078], length 19432: HTTP

07:18:26.457422 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq
2356824:2373480, ack 1, win 235, options [nop,nop,TS val 2226567777 ecr
3054281682], length 16656: HTTP
```

07:18:26.746414 IP 185.117.80.53.80 > 192.168.124.137.60492: Flags [.], seq 964376218:964395650, ack 1034282473, win 235, options [nop,nop,TS val 2226568063 ecr 3054281963], length 19432: HT

TP

07:18:26.978560 IP 185.117.80.53.80 > 192.168.124.137.60502: Flags [.], seq 3786023752:3786039020, ack 3169565691, win 235, options [nop,nop,TS val 2226568298 ecr 3054282201], length 15268:

HTTP

07:18:27.195761 IP 185.117.80.53.80 > 192.168.124.137.60492: Flags [.], seq 570468:589900, ack 1, win 235, options [nop,nop,TS val 2226568513 ecr 3054282418], length 19432: HTTP

07:18:27.391916 IP 185.117.80.53.80 > 192.168.124.137.60492: Flags [.], seq 831412:848068, ack 1, win 235, options [nop,nop,TS val 2226568707 ecr 3054282611], length 16656: HTTP

Displaying Packets

Customizing how Tcpdump displays captured packets can make analysis more straightforward:

- tcpdump -q: Quick output with brief information.
- tcpdump -e: Displays MAC addresses.

- tcpdump -A: Shows packet contents in ASCII format.
- tcpdump -xx: Displays packet data in hexadecimal format.
- tcpdump -X: Shows both hexadecimal and ASCII formats.

Example:

```
user@TryHackMe$ tcpdump -r TwoPackets.pcap -X

reading from file TwoPackets.pcap, link-type EN10MB (Ethernet), snapshot length
262144

18:59:59.979771 IP 104.18.12.149.https > g5000.45248: Flags [P.], seq
2695955324:2695955349, ack 2856007037, win 16, options [nop,nop,TS val
412758285 ecr 3959057198], length 25

        0x0000:  4500 004d fbd8 4000 3506 d229 6812 0c95  E..M..@.5..)h...

        0x0010:  c0a8 4259 01bb b0c0 a0b1 037c aa3b 357d  ..BY.......|.;5}

        0x0020:  8018 0010 f905 0000 0101 080a 189a 310d  .............1.

        0x0030:  ebfa 6b2e 1703 0300 146a 8f33 1832 e6a2  ..k......j.3.2..

        0x0040:  fb99 eb26 3961 dad4 1611 152d 4c         ...&9a.....-L
```

18:59:59.980574 IP g5000.45248 > 104.18.12.149.https: Flags [P.], seq 1:30, ack 25, win 2175, options [nop,nop,TS val 3959057384 ecr 412758285], length 29

```
0x0000:  4500 0051 6ca8 4000 4006 5656 c0a8 4259  E..Ql.@.@.VV..BY

0x0010:  6812 0c95 b0c0 01bb aa3b 357d a0b1 0395  h........;5}....

0x0020:  8018 087f 17e0 0000 0101 080a ebfa 6be8  ..............k.

0x0030:  189a 310d 1703 0300 18f4 31fa 798d 2656  ..1.......1.y.&V

0x0040:  433c 2389 5f4a 24c2 fa7a 1496 8444 238e  C<#._J$..z...D#.

0x0050:  60
```

Answer the questions below

What is the MAC address of the host that sent an ARP request?

Answer: 52:54:00:7c:d3:5b

```
user@ip-10-10-218-40:~$ sudo tcpdump -r traffic.pcap arp -e

reading from file traffic.pcap, link-type EN10MB (Ethernet)
```

07:18:29.940761 52:54:00:7c:d3:5b (oui Unknown) > Broadcast, ethertype ARP (0x0806), length 42: Request who-has ip-192-168-124-137.eu-west-1.compute.internal tell ip-192-168-124-148.eu-west-

1.compute.internal, length 28

07:18:29.940776 52:54:00:23:60:2b (oui Unknown) > 52:54:00:7c:d3:5b (oui Unknown), ethertype ARP (0x0806), length 42: Reply ip-192-168-124-137.eu-west-1.compute.internal is-at 52:54:00:23:60

:2b (oui Unknown), length 28

My Takeaway:

Using the `-A` flag allowed me to see the raw text within packets. I was able to see clear-text HTTP credentials and user-agents, highlighting why **unencrypted protocols (HTTP/Telnet/FTP)** are a major security risk.

💡 Why This Matters in Cybersecurity

- **Remote Forensics:** If a server is compromised, `tcpdump` is often the first tool used to see what the attacker is doing in real-time.
- **Automation:** Because it's CLI-based, `tcpdump` can be scripted to trigger captures based on specific events (e.g., a spike in traffic).
- **Efficiency:** It uses significantly less RAM than a GUI tool, making it safer to run on critical production systems.